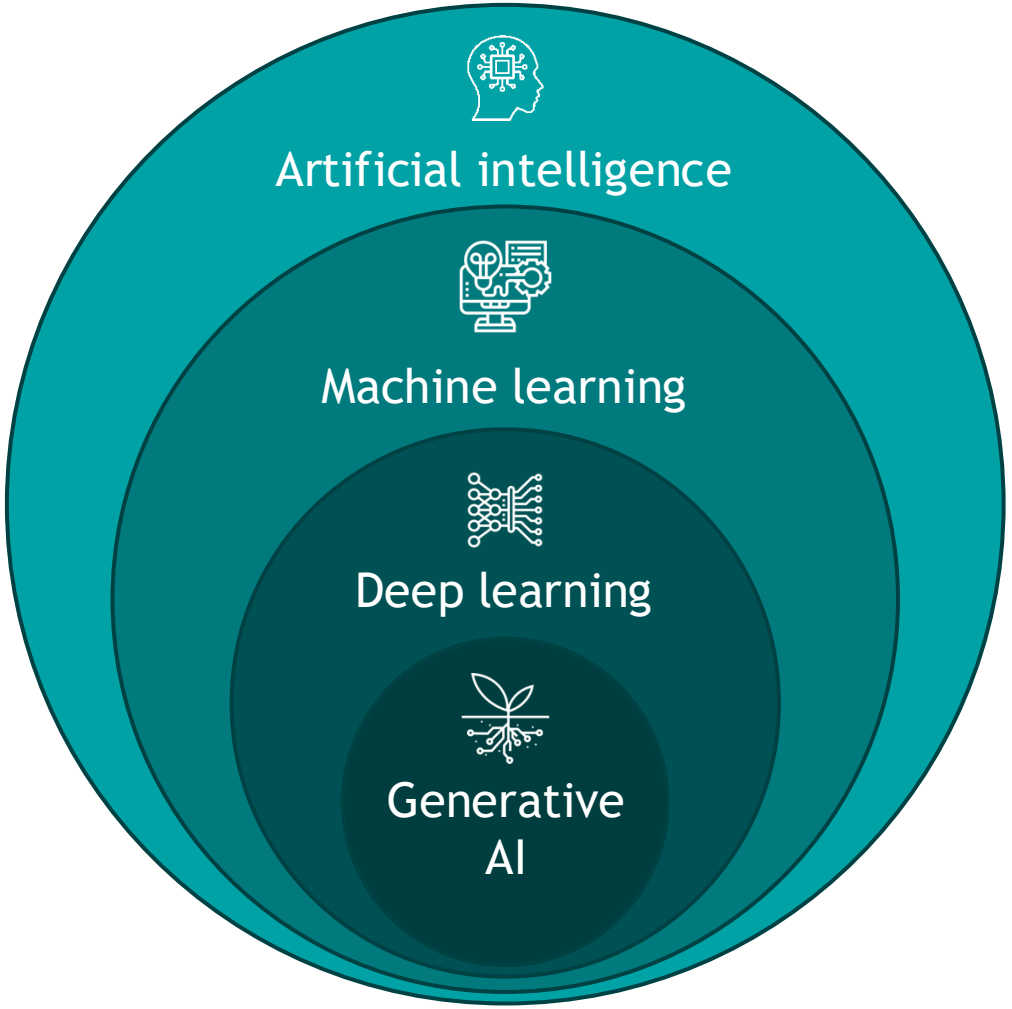
01 Introduction to Neural Networks

Why deep learning?

Building blocks of neural networks

Network architecture

Training neural networks

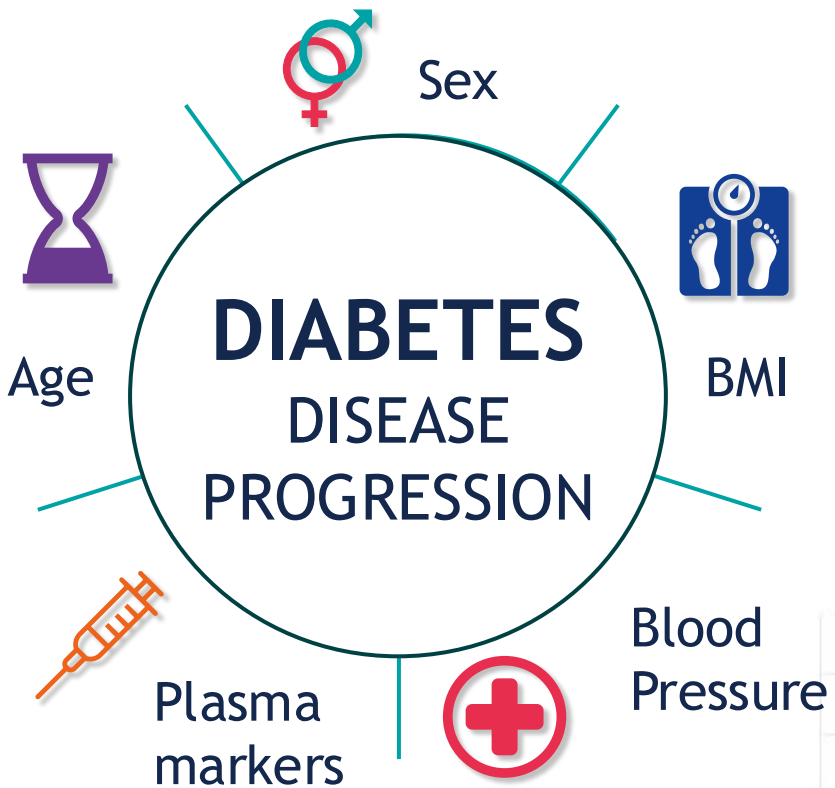


Machine learning – system learns the rules from previous data

Deep learning – machine learning with deep neural networks

→ Same principles as machine learning, but with a specific model architecture

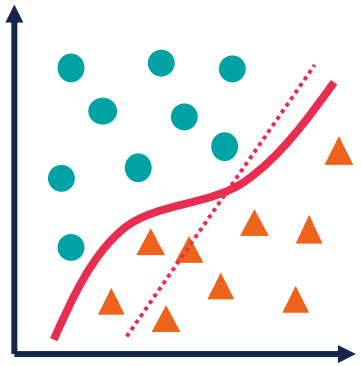
VIB



Feature engineering

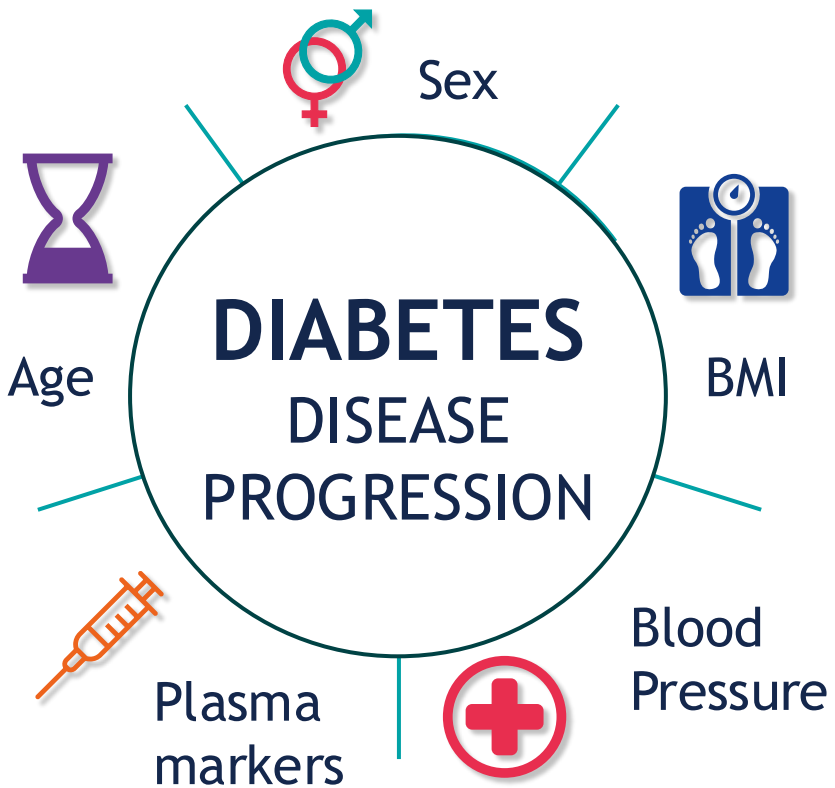
= transforming raw data into meaningful input variables (features)

ML model



ECG features

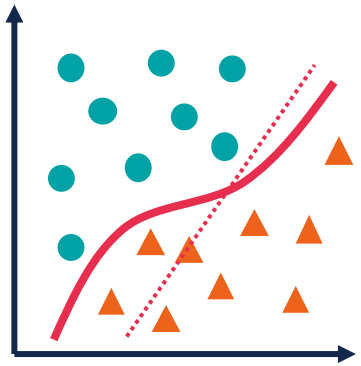
- Requires domain knowledge (assumptions!)
- Complex/invisible patterns?



Feature engineering

= transforming raw data into meaningful input variables (features)

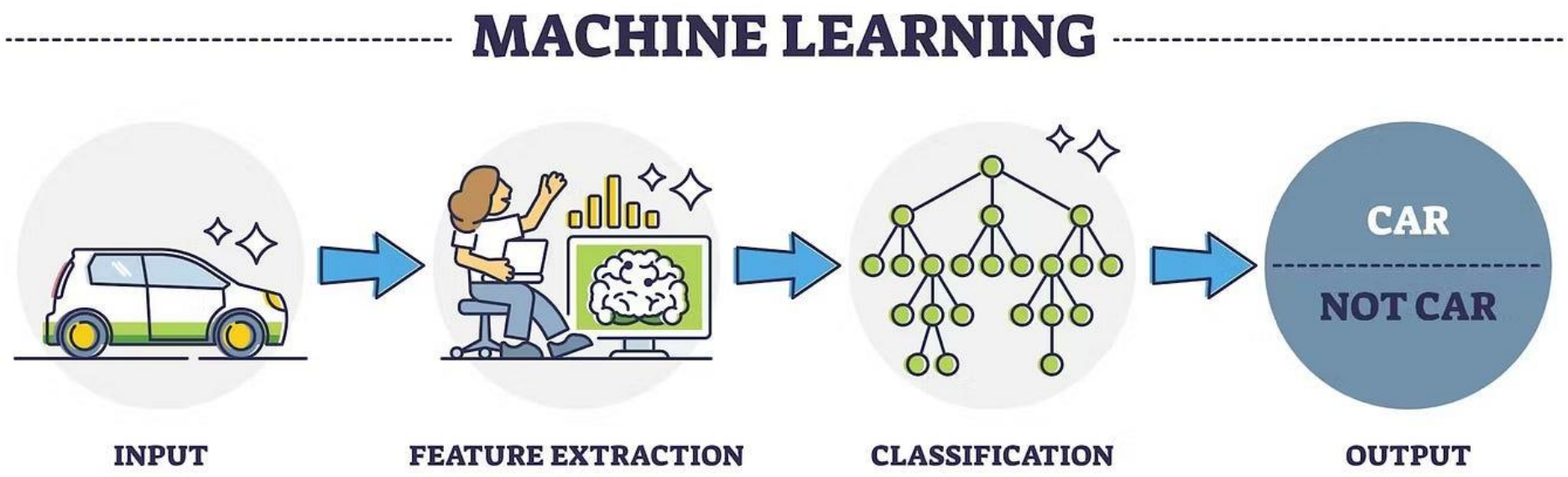
ML model



CT features
(e.g. fat distribution)

- Too difficult to compute
- Too expensive to label

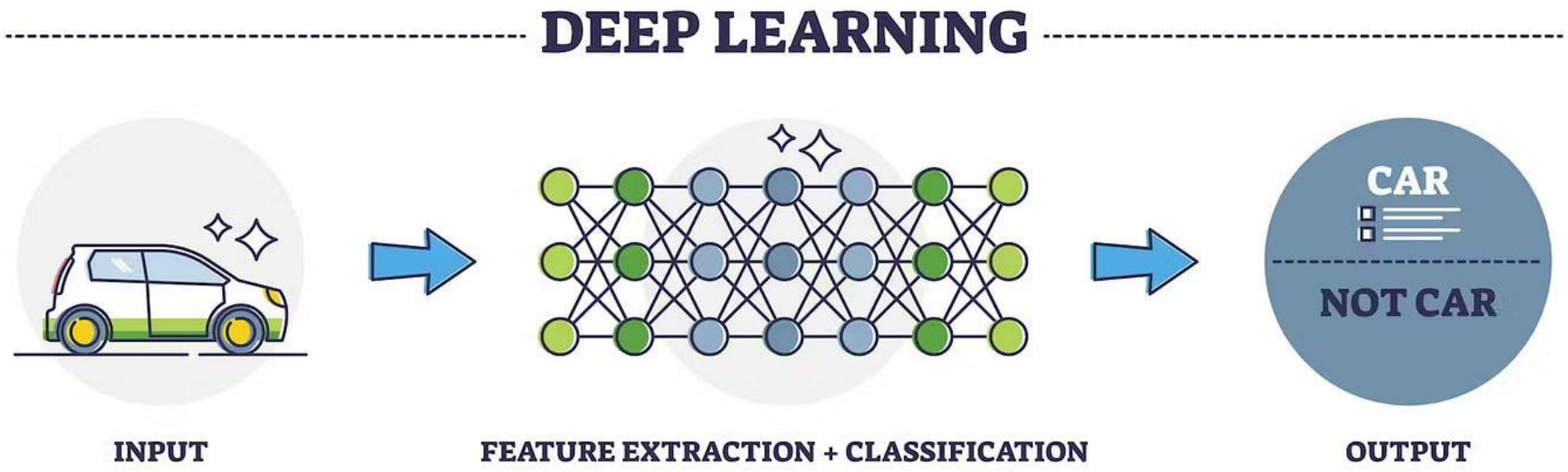
VIB



You decide what information the model gets

- Requires feature engineering
- Can work with thousands of examples
- High interpretability
- Often best choice for tabular data

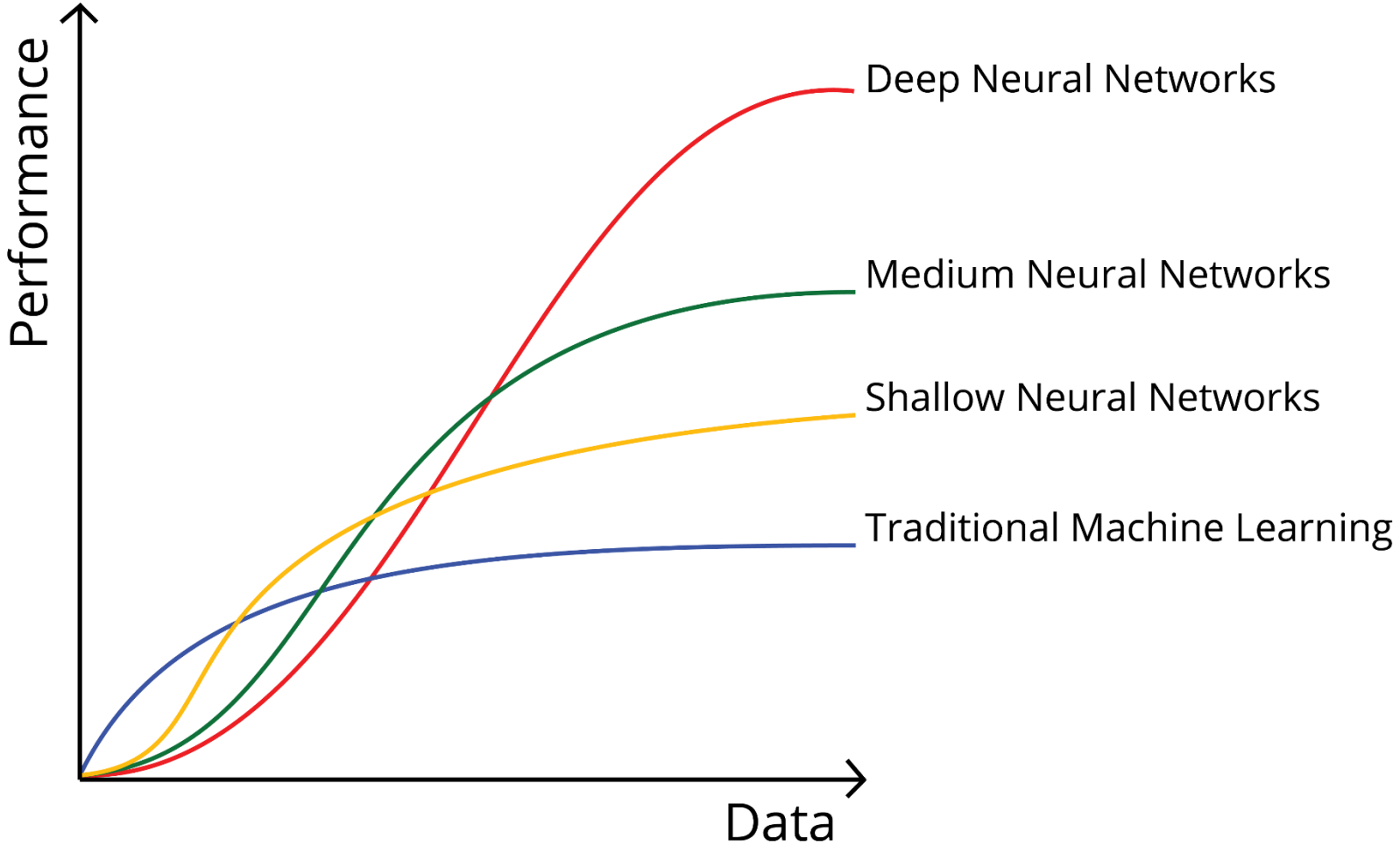
Image adapted from: https://medium.com/@jainpalak9509/why-deep-learning-over-traditional-ml-algorithms-efae1660f3b0



Neural networks learn their own features

- (often) needs millions of examples
- High training cost
- Low interpretability
- Best choice for raw/unstructured data

Image adapted from: https://medium.com/@jainpalak9509/why-deep-learning-over-traditional-ml-algorithms-efae1660f3b0


Performance
Deep Neural Networks
Medium Neural Networks
Shallow Neural Networks
Traditional Machine Learning
Data

Why deep learning?

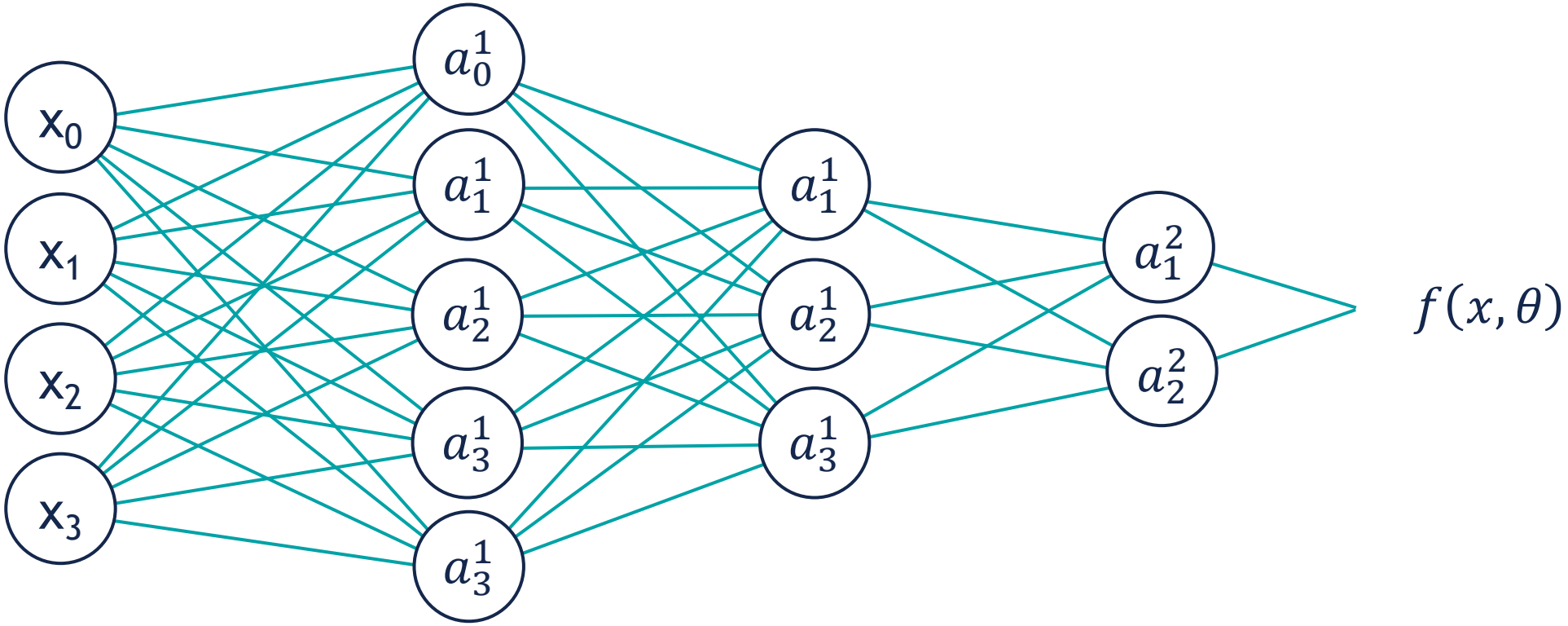
Building blocks of neural networks

Network architecture

Training neural networks

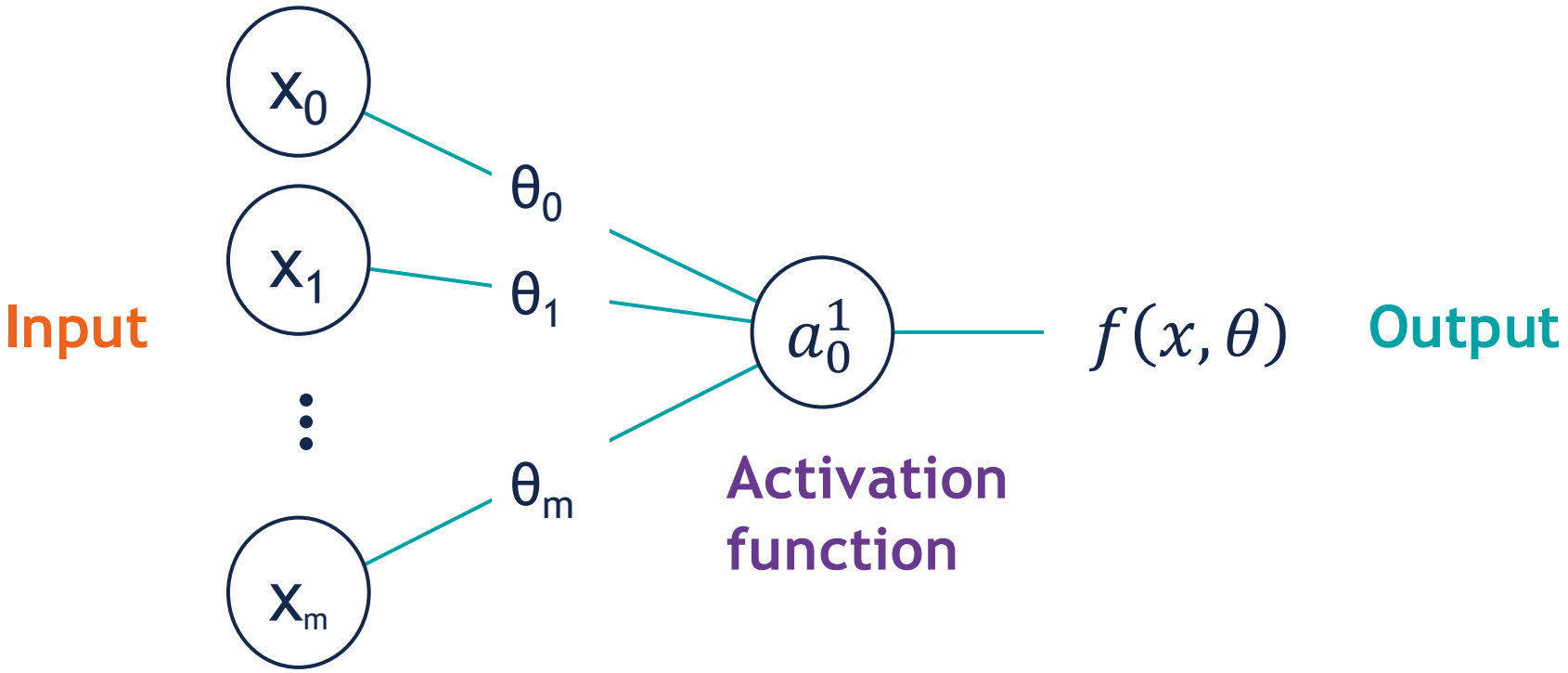
Neural network

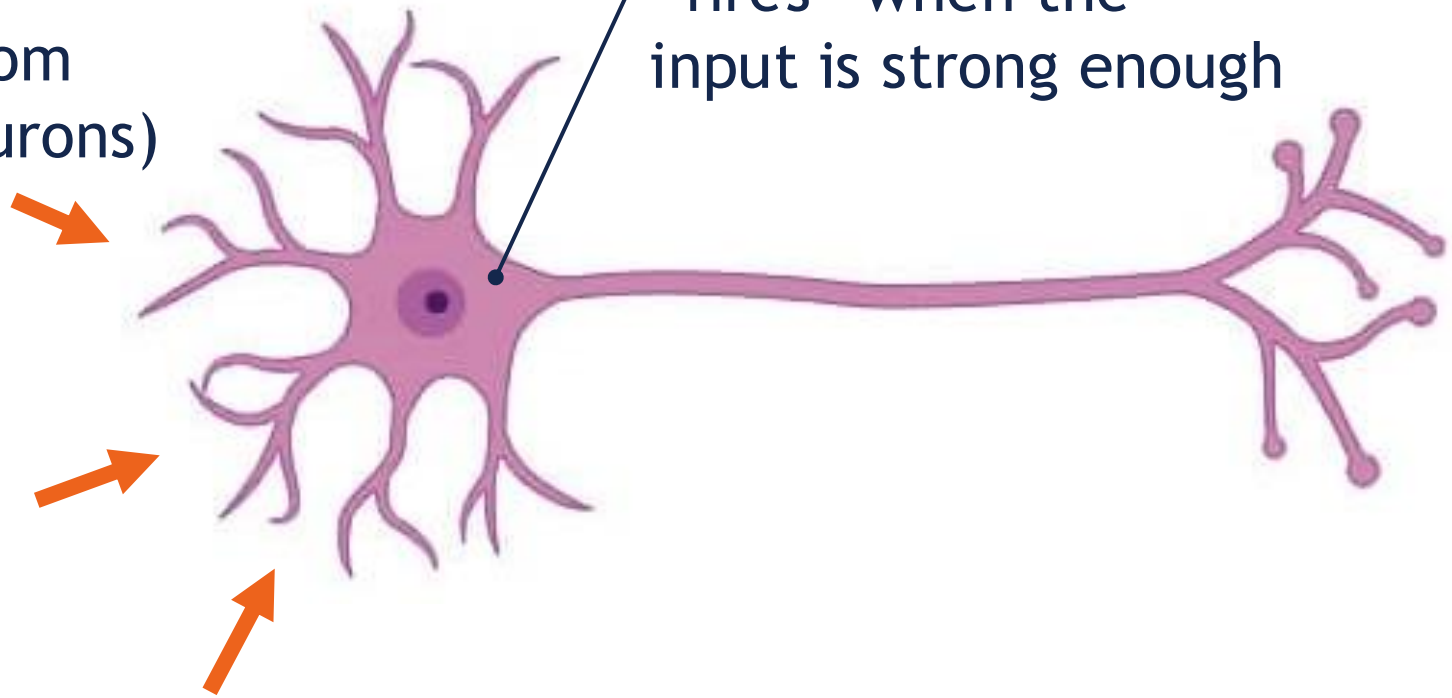
= an architecture that consists of many perceptrons



Perceptron

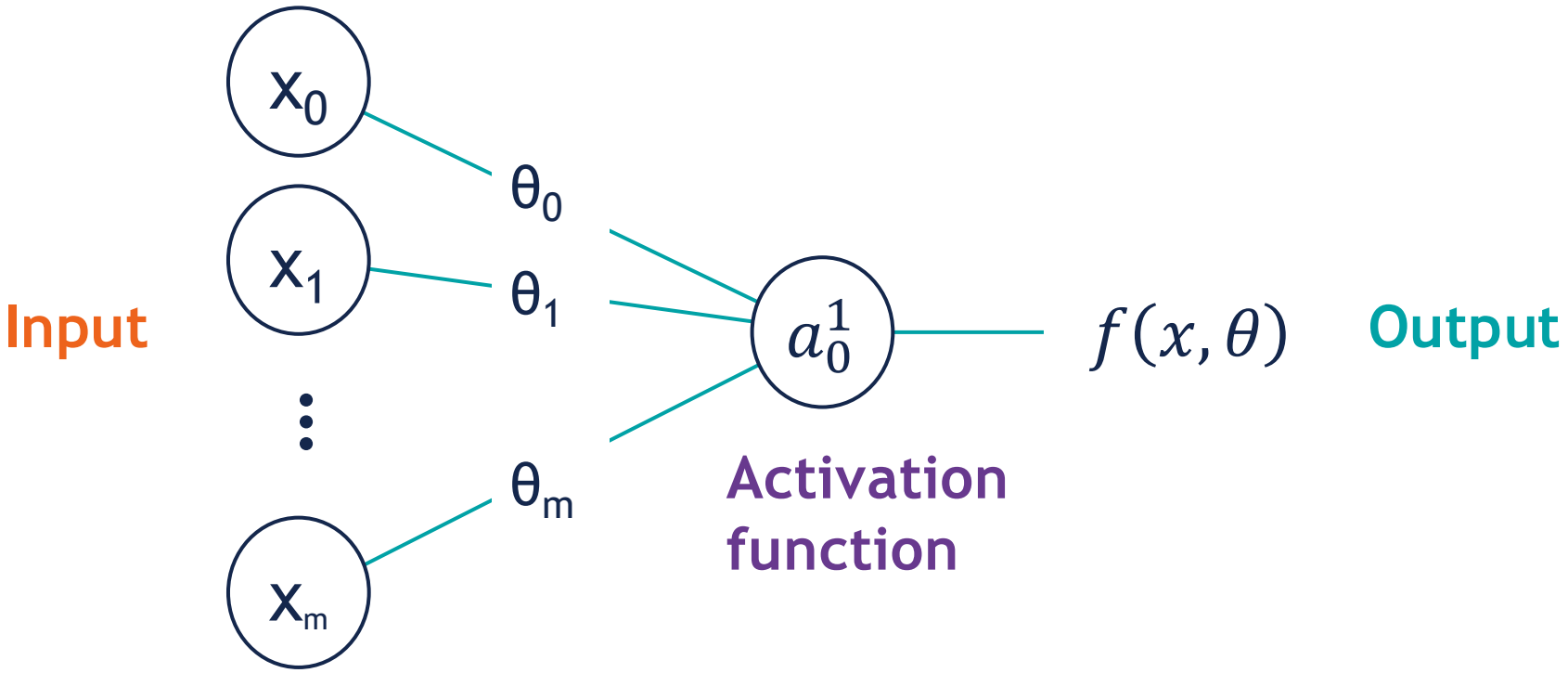
= building block of a neural network




Cell body
“fires” when the input is strong enough
Input (from other neurons)
Output (to other neurons)

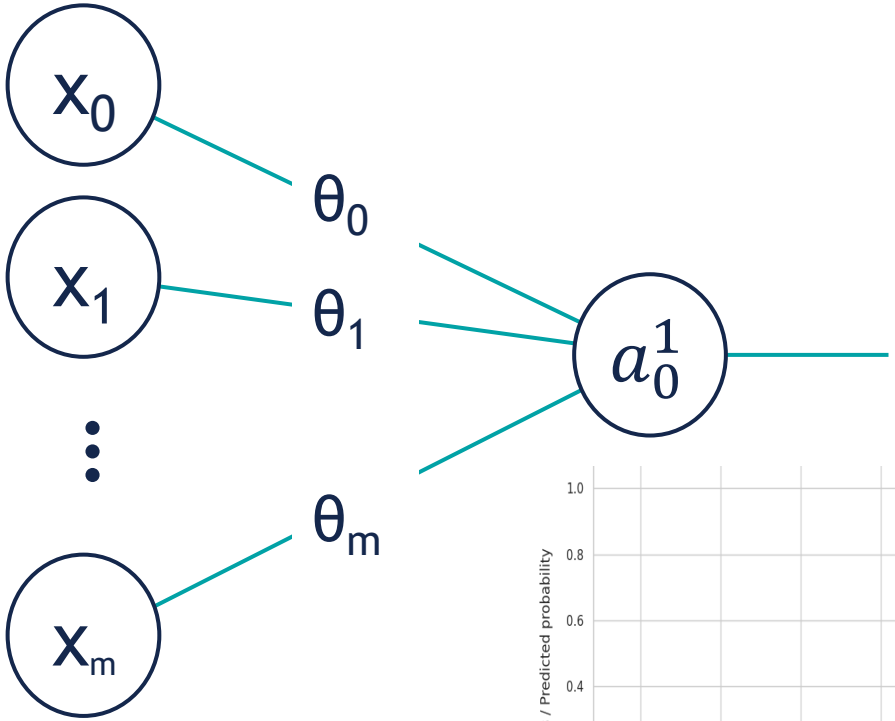
Perceptron

= building block of a neural network

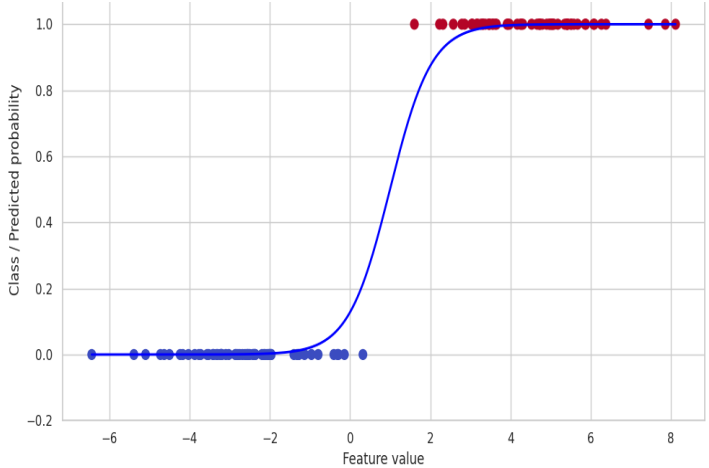


Perceptron

= building block of a neural network



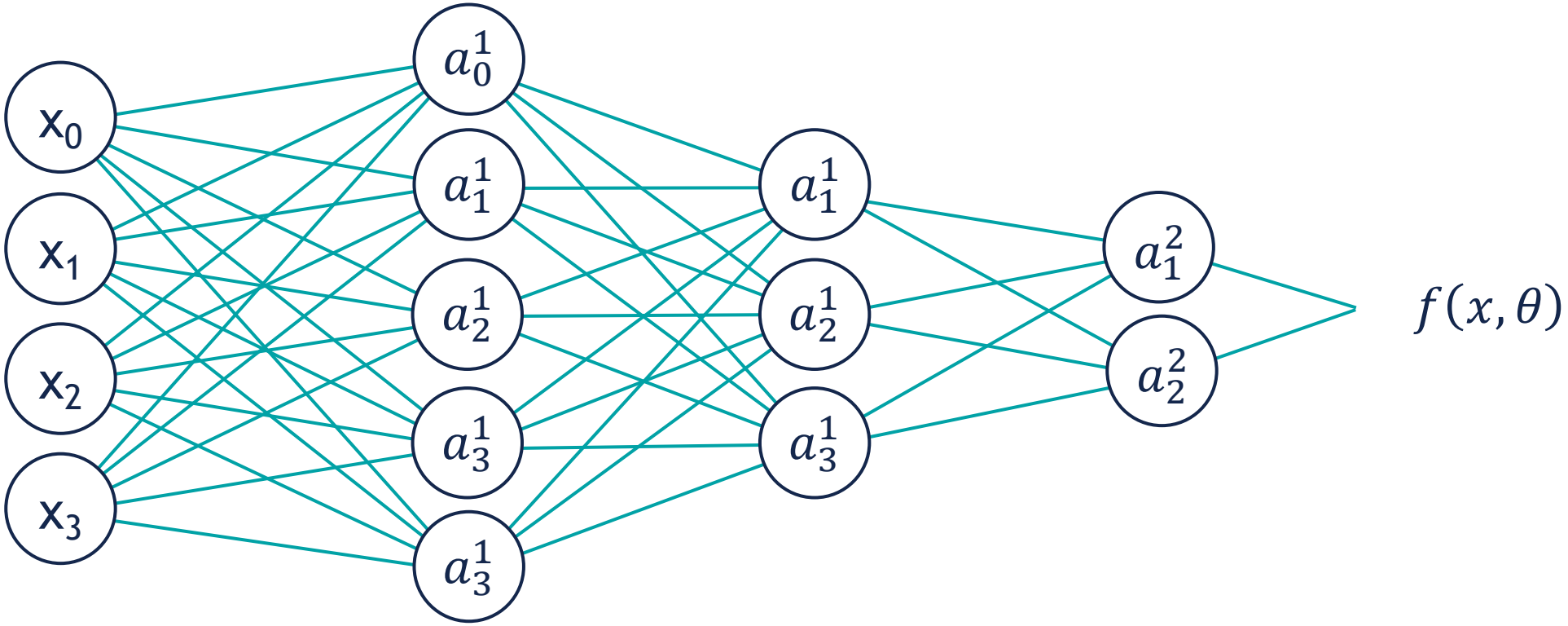
$$f(x,\theta) = a(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_m x_m)$$



- Linear combination
  → Linear regression
- (sigmoid) activation function
  → Logistic regression

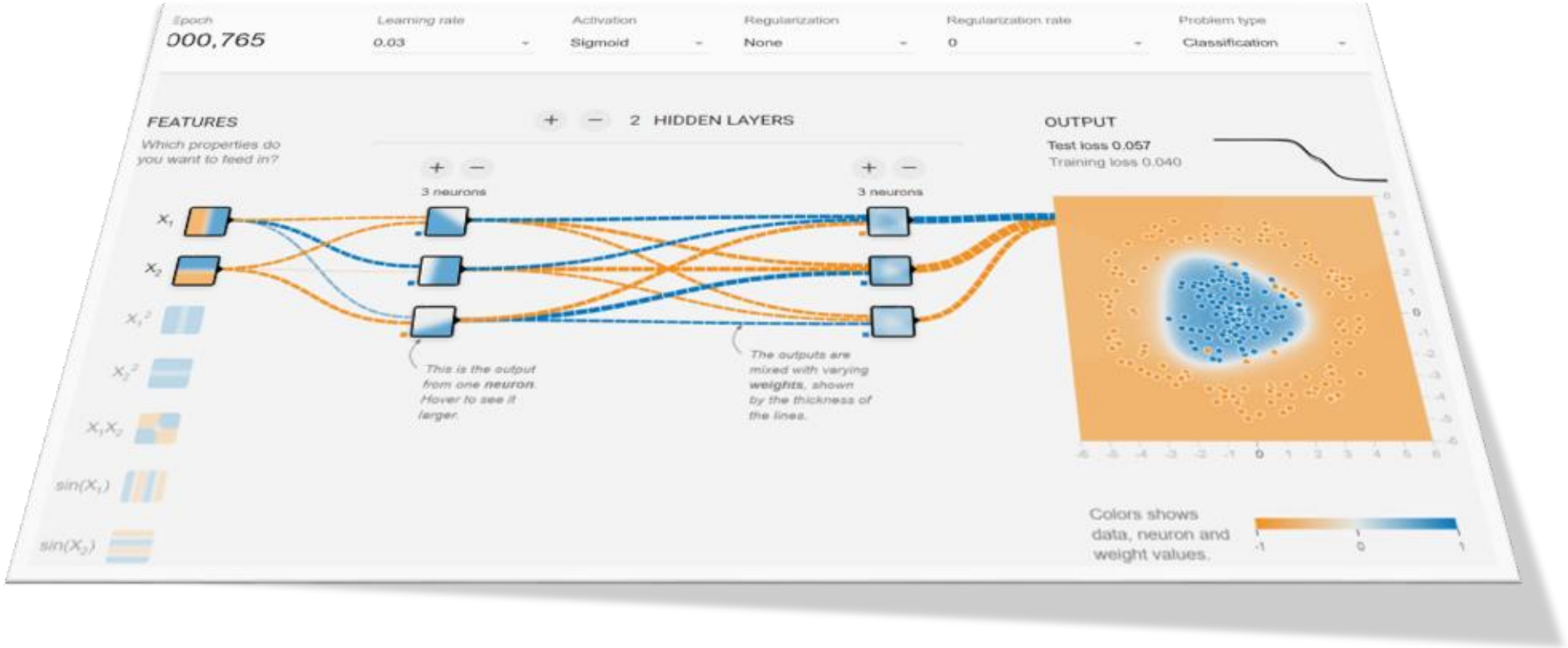
Neural network

= an architecture that consists of many perceptrons



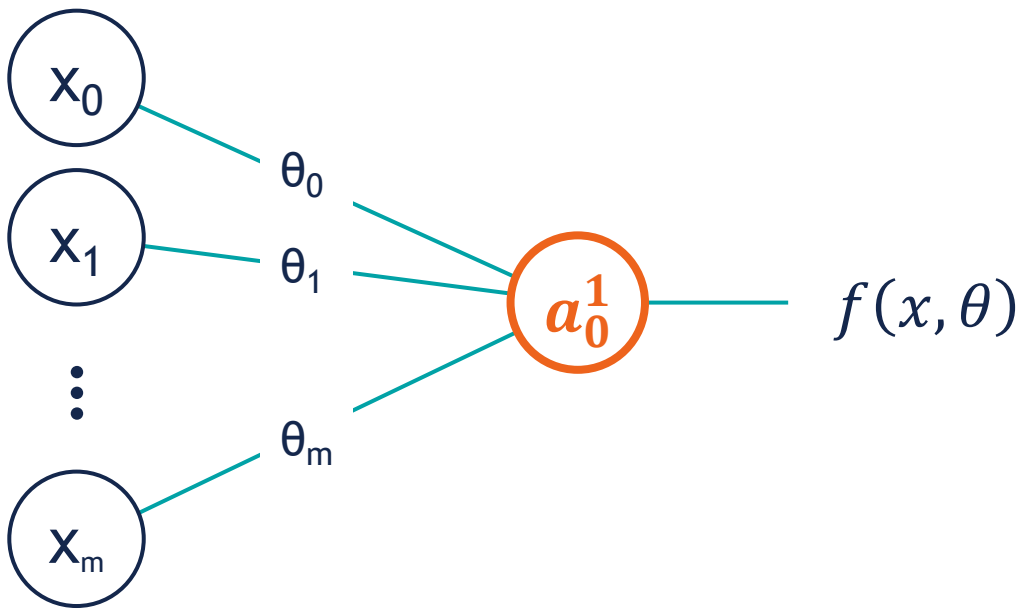
Neural network

Demo: create a small neural network



https://playground.tensorflow.org/

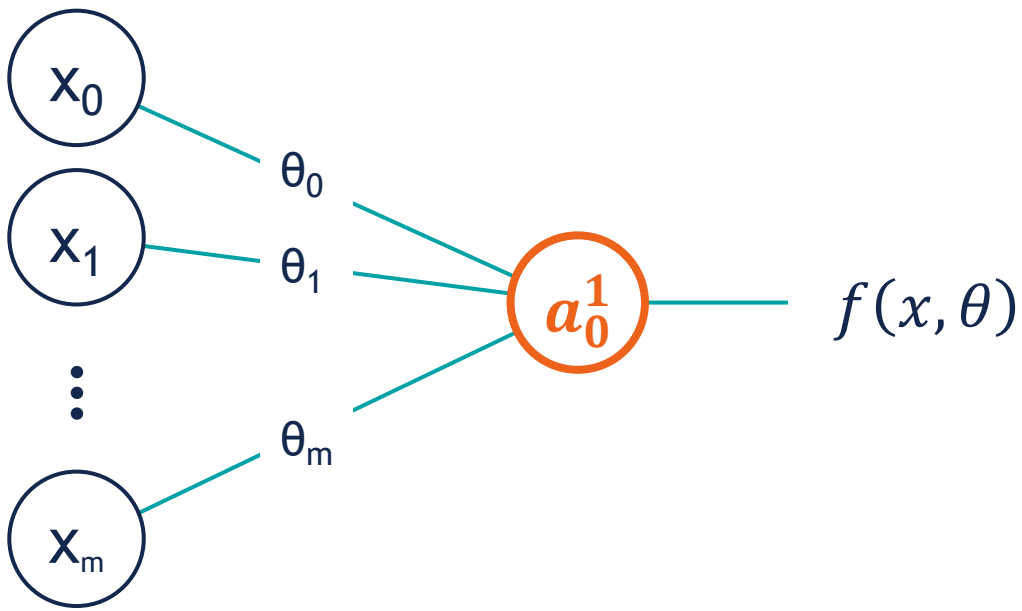
Activation functions



What happens if we use a linear activation function?

A combination of linear regressors = linear regressor

Activation functions

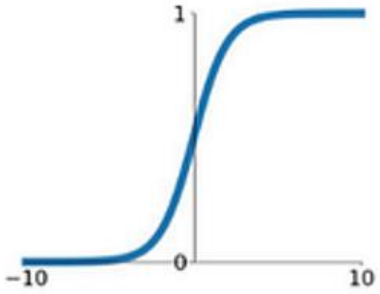


- Introduces non-linearity in the system
- Necessary to learn complex patterns in the data
- Popular functions: sigmoid, tanh, ReLU

Activation functions

Sigmoid

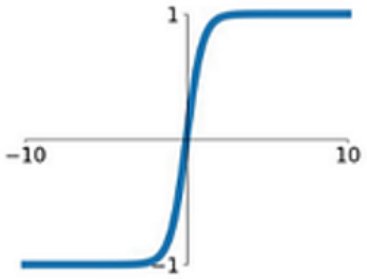
$$\sigma(x) = \frac{1}{1+e^{-x}}$$



"Mimics a neuron firing"

tanh

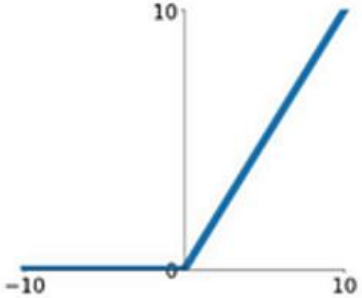
$$\tanh(x)$$



"Scaled sigmoid", zero-centered

ReLU

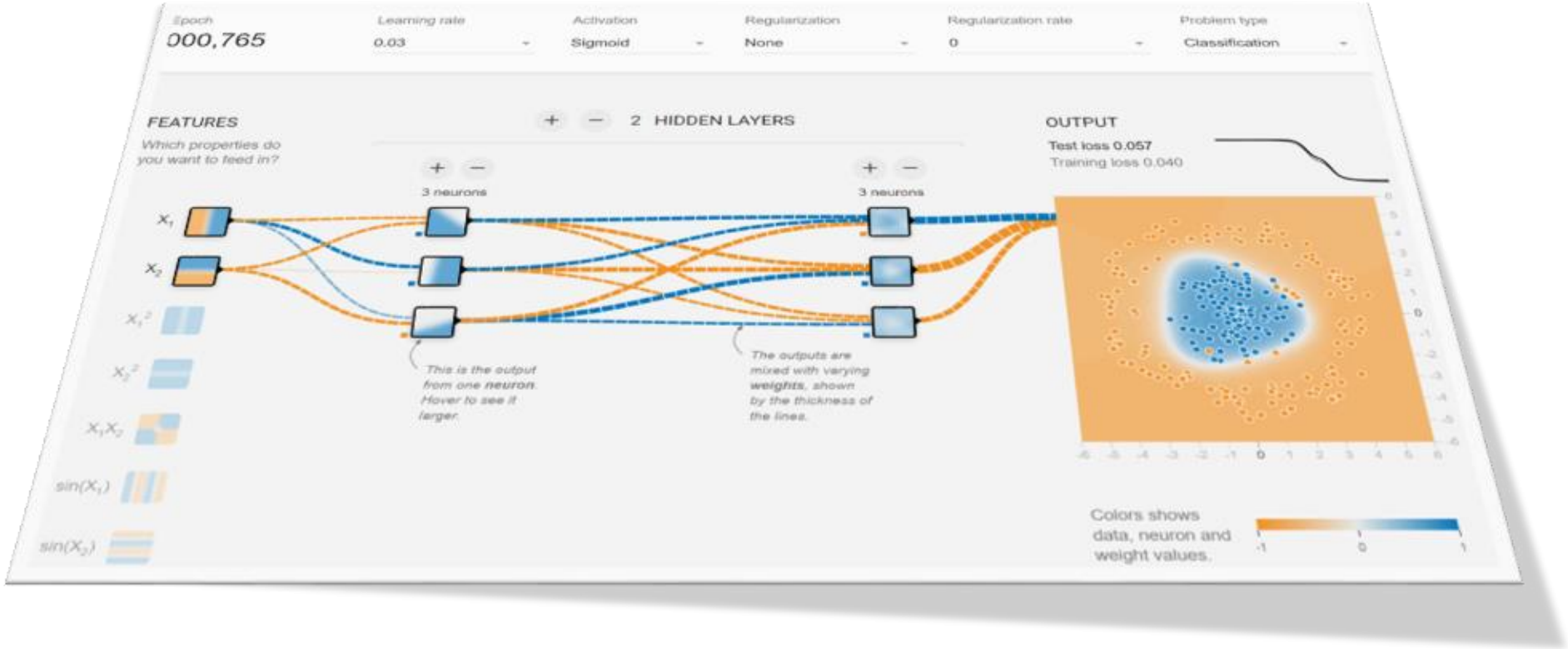
$$\max(0, x)$$



Simple to compute, but not zero-centered

Activation functions

Non-linear activation functions increase the expressiveness of the model



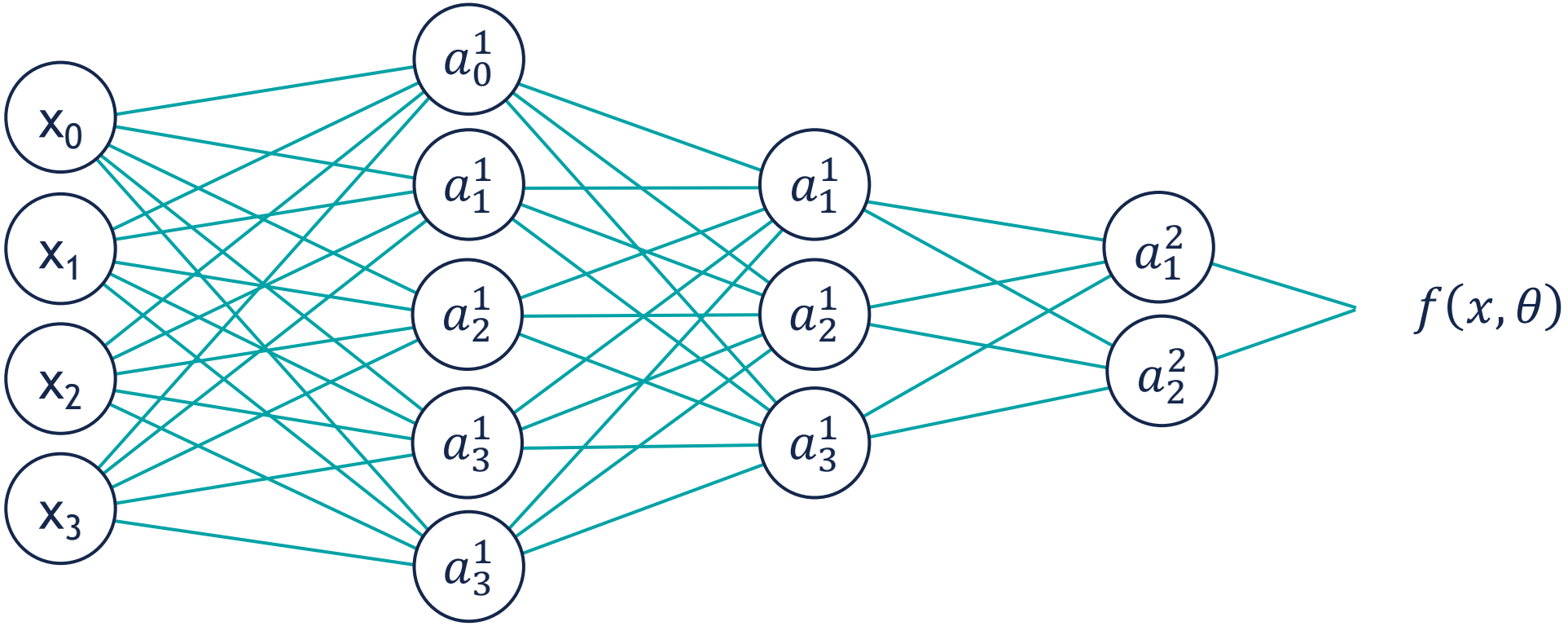
Why deep learning?

Building blocks of neural networks

Network architecture

Training neural networks

Network architecture

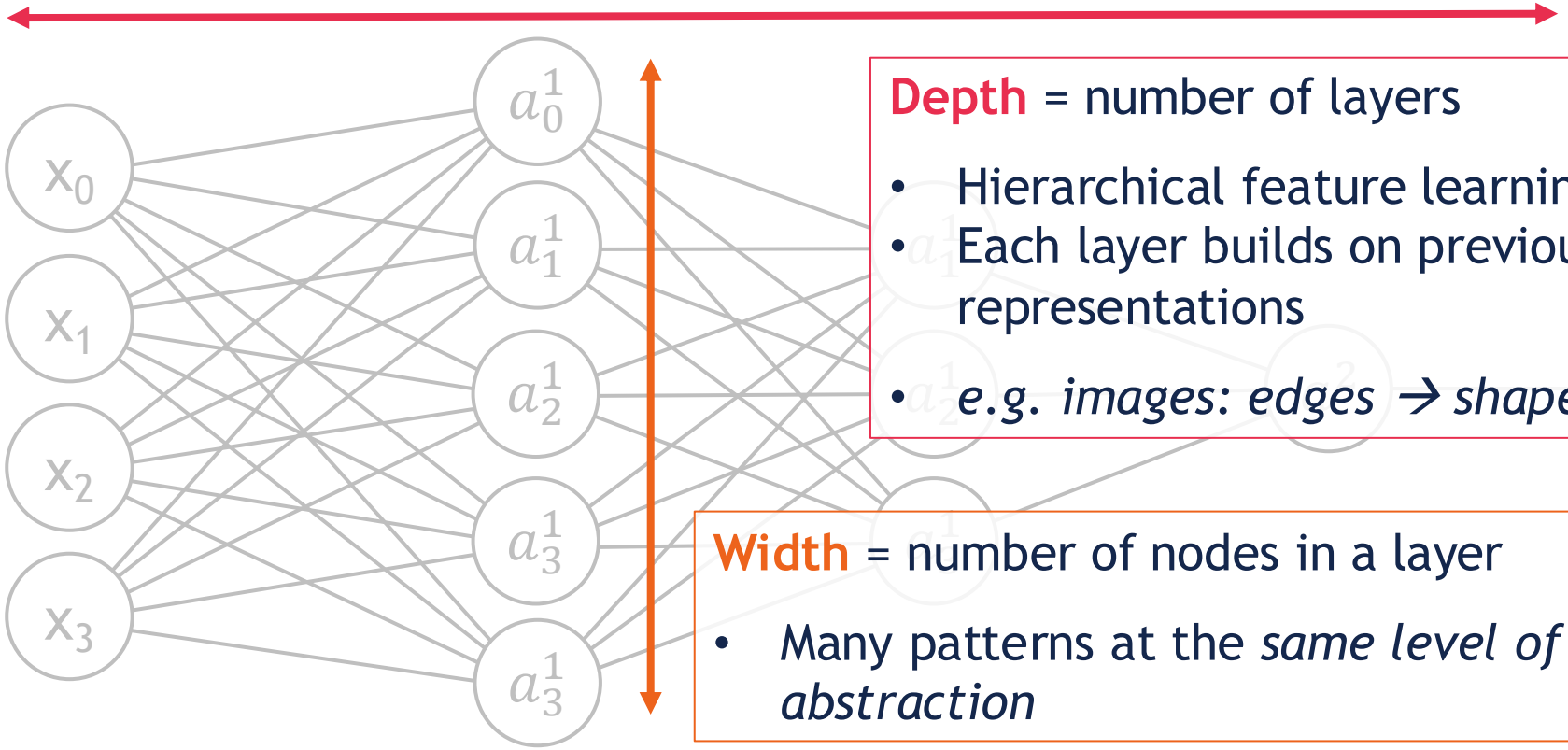


Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

Network architecture



Depth = number of layers

- Hierarchical feature learning
- Each layer builds on previous representations
- *e.g. images: edges → shapes → objects*

Width = number of nodes in a layer

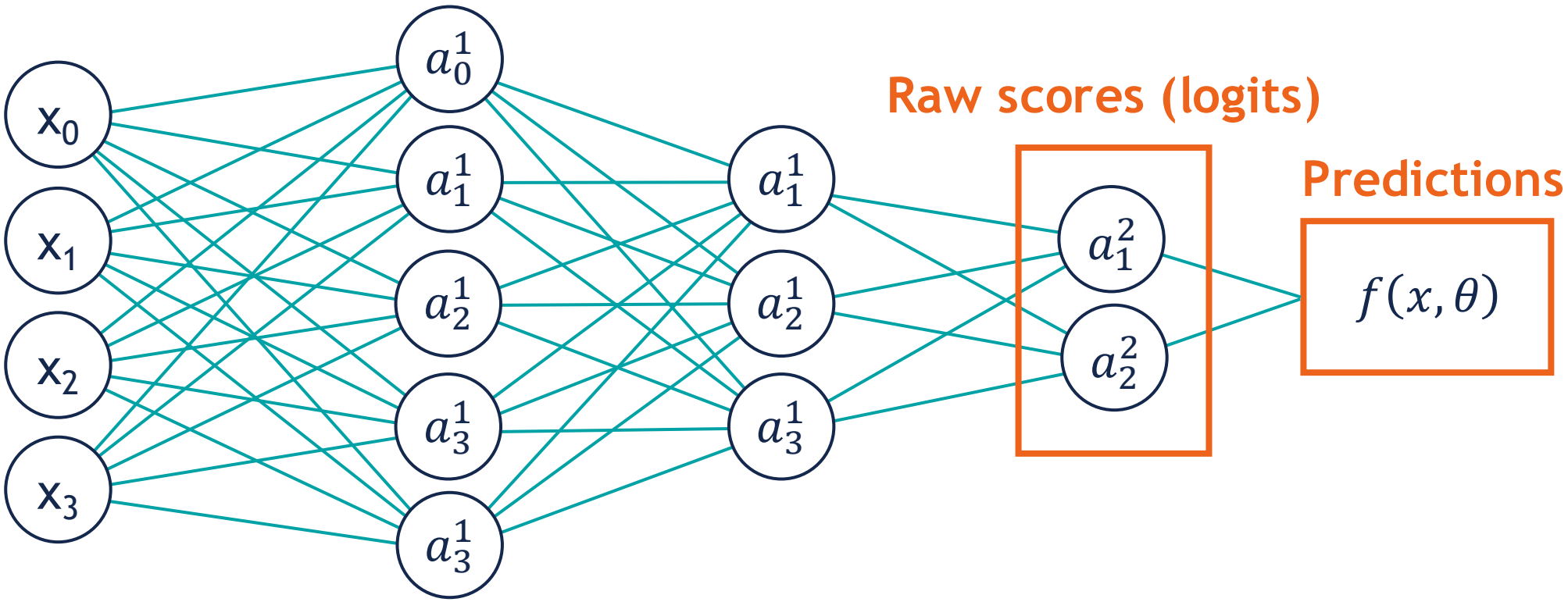
- Many patterns at the *same level of abstraction*

Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

Output layer



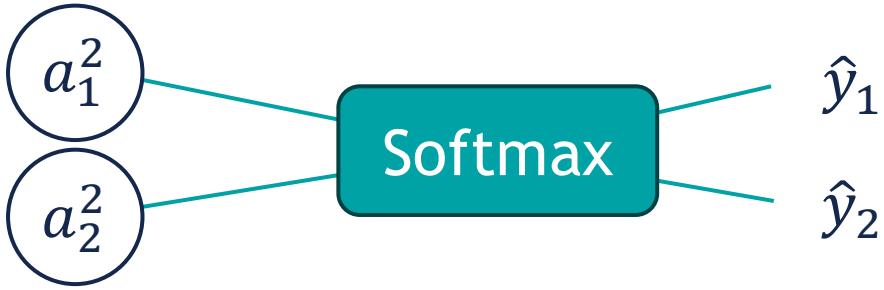
Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

VIB

Softmax

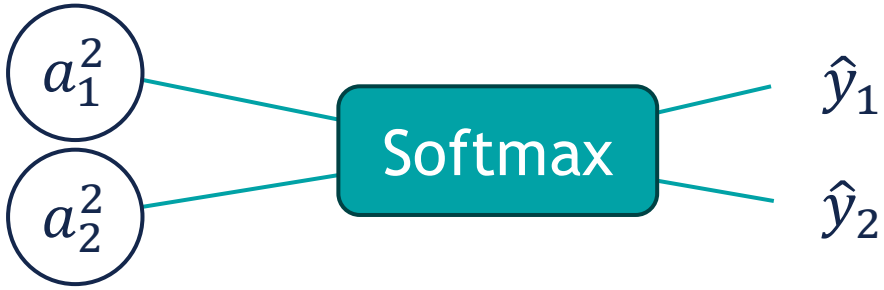


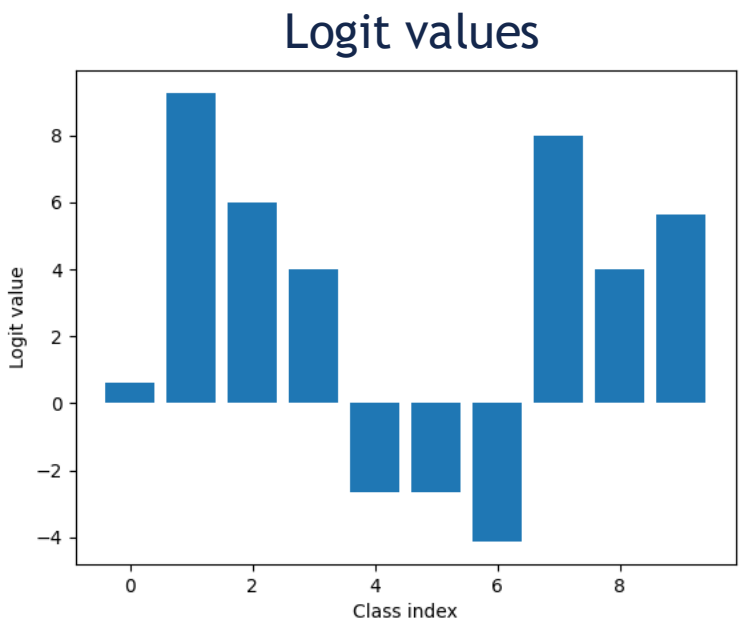
- Softmax function transforms logits to confidence scores

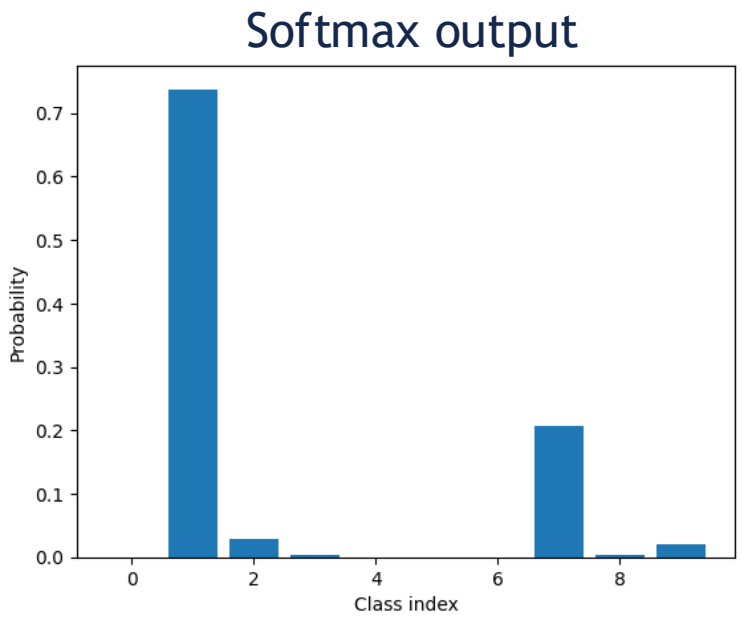
$$\sigma(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Exponentials exaggerate the differences
- Normalization produces probabilities

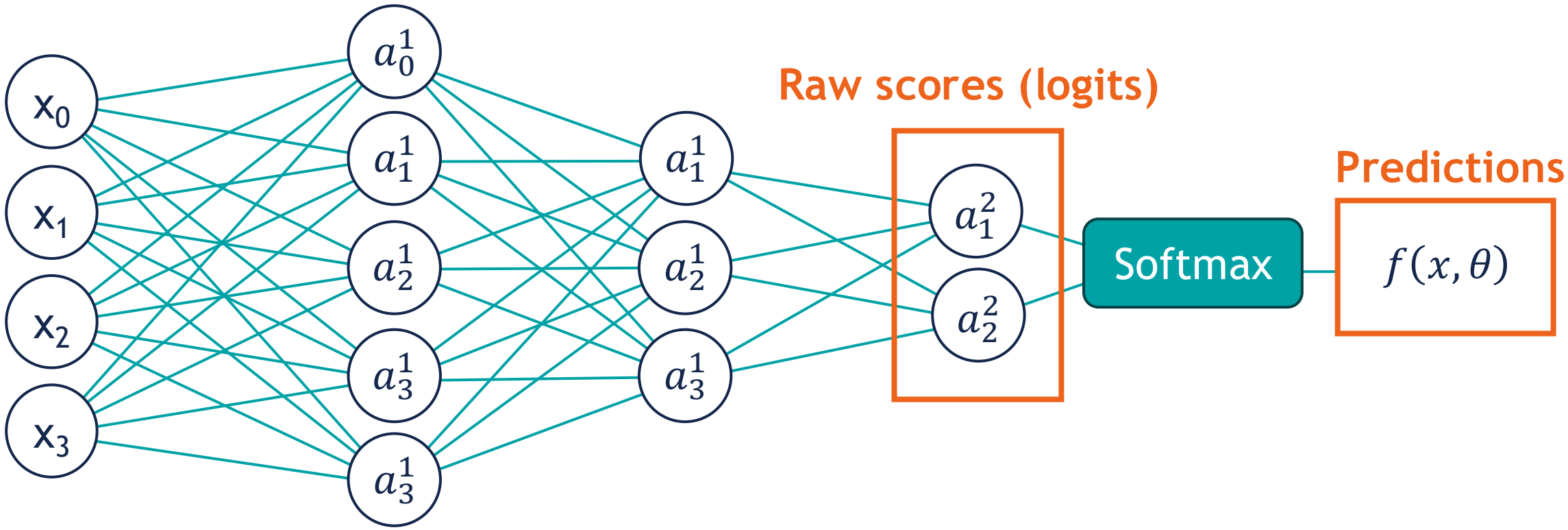
Softmax


a_1^2
a_2^2
Softmax
$\hat{y}_1$
$\hat{y}_2$


Logit values
Logit value
Class index
8
6
4
2
0
−2
−4
0
2
4
6
8


Softmax output
Probability
Class index
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0
2
4
6
8

Output layer



Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

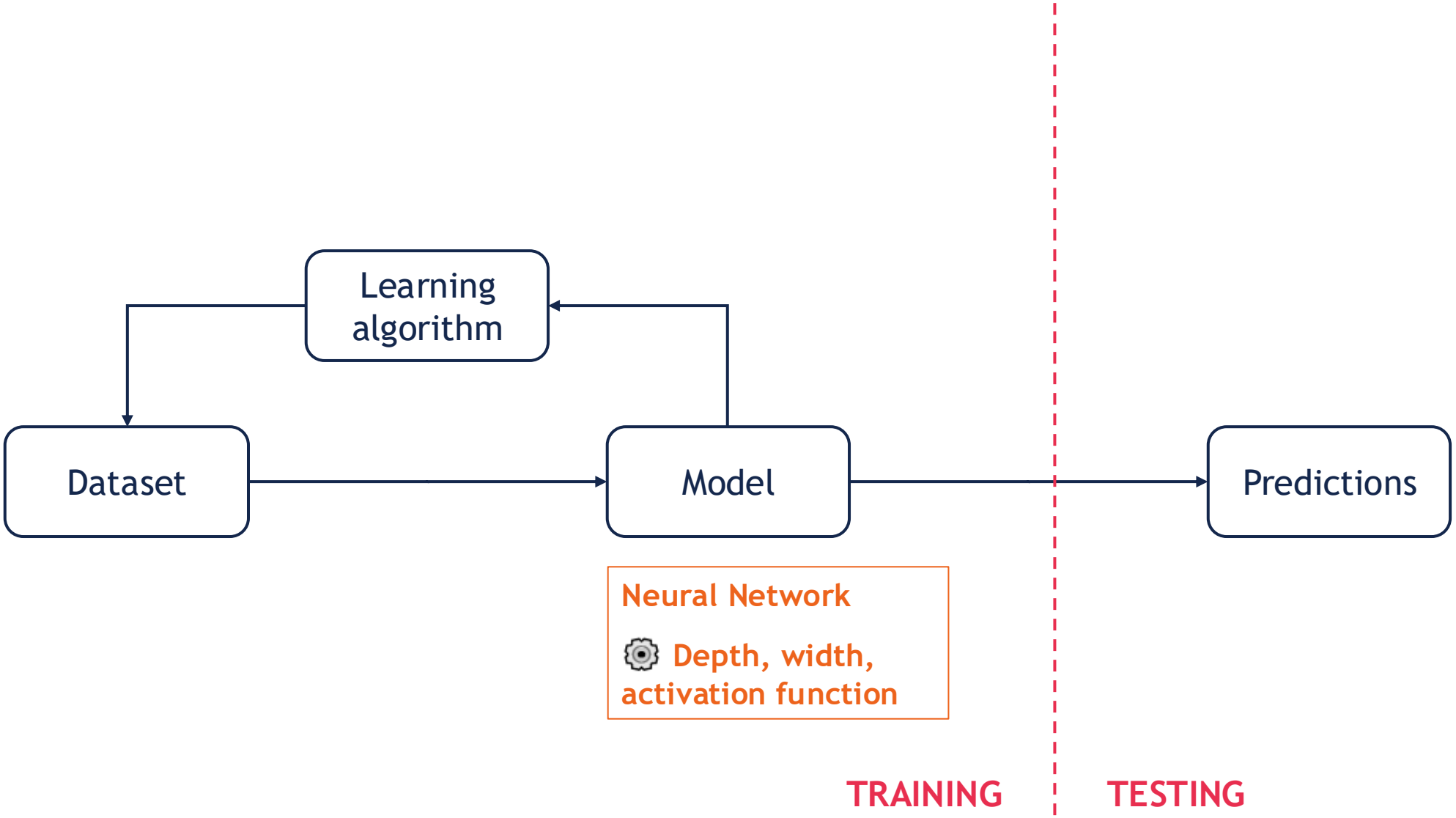
Why deep learning?

Building blocks of neural networks

Network architecture

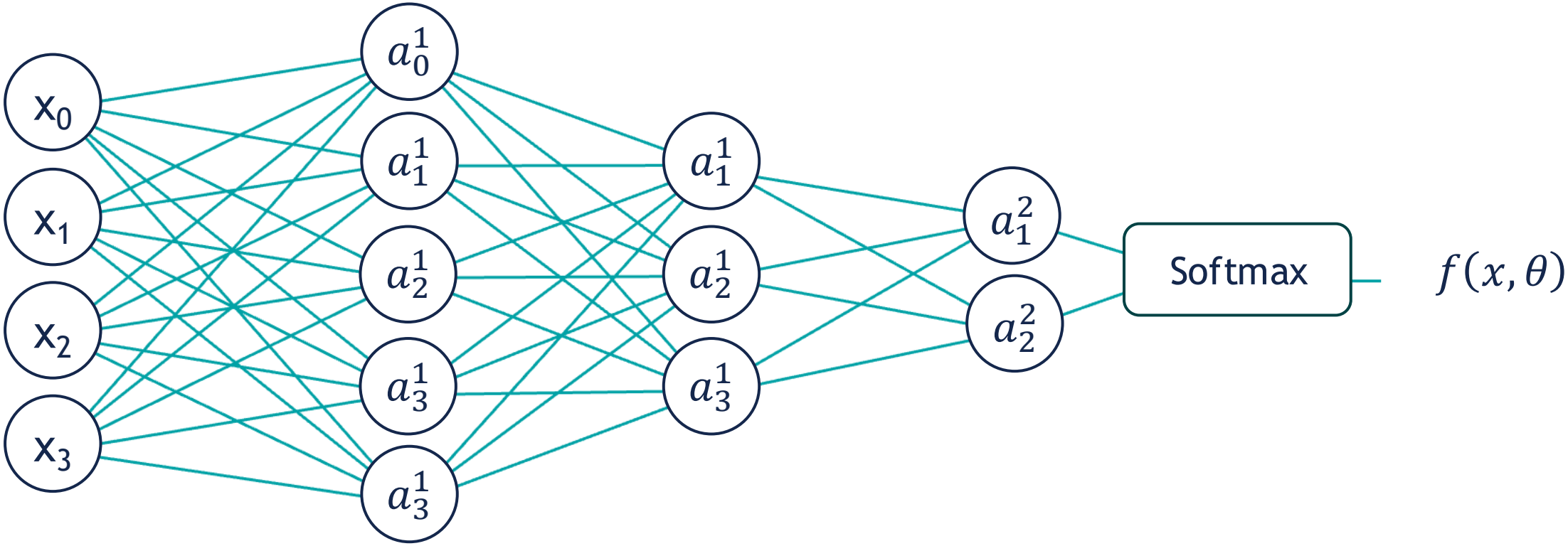
Training neural networks

Training a neural network



Forward pass

Compute the predictions based on the features (output from layer N is the input for layer N+1)



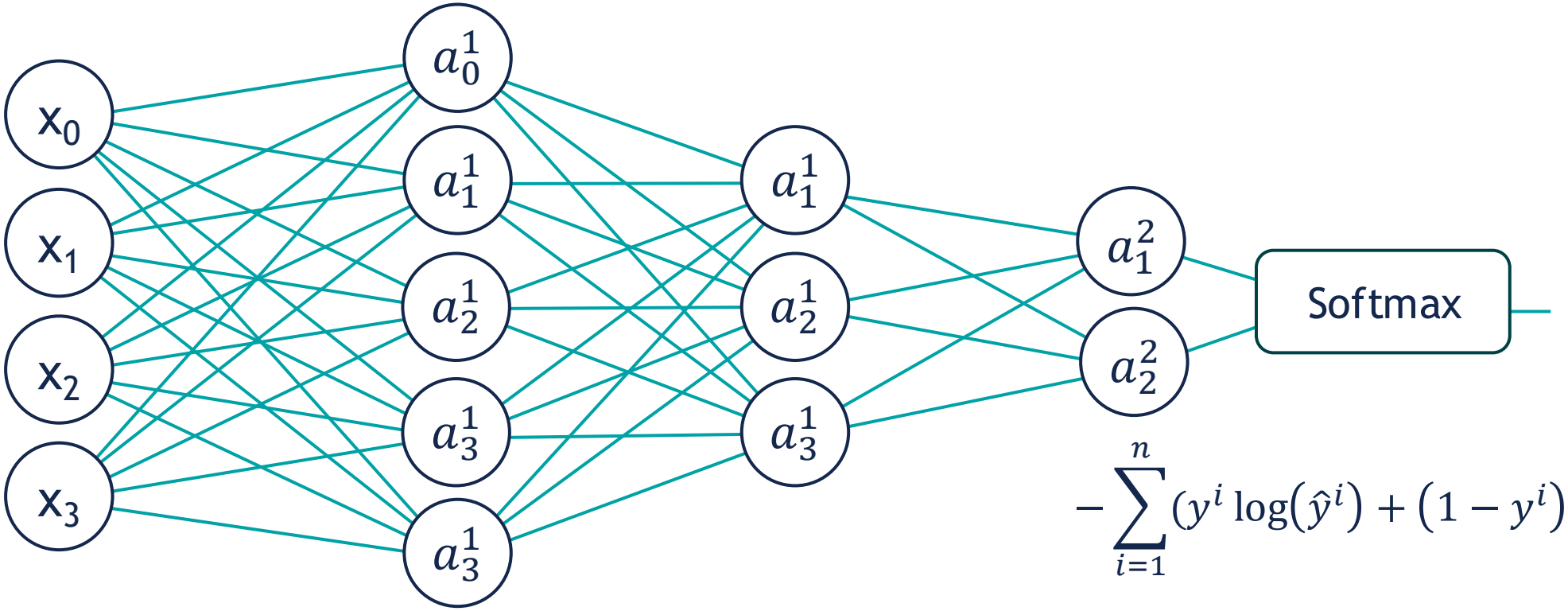
Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

Loss function

Cross entropy quantifies how far the predictions are from the true values



$$-\sum_{i=1}^{n}\left(y^i \log(\hat{y}^i) + (1-y^i)\log(1-\hat{y}^i)\right)$$

Input
(4 features)

Hidden layers
(5 and 3 nodes)

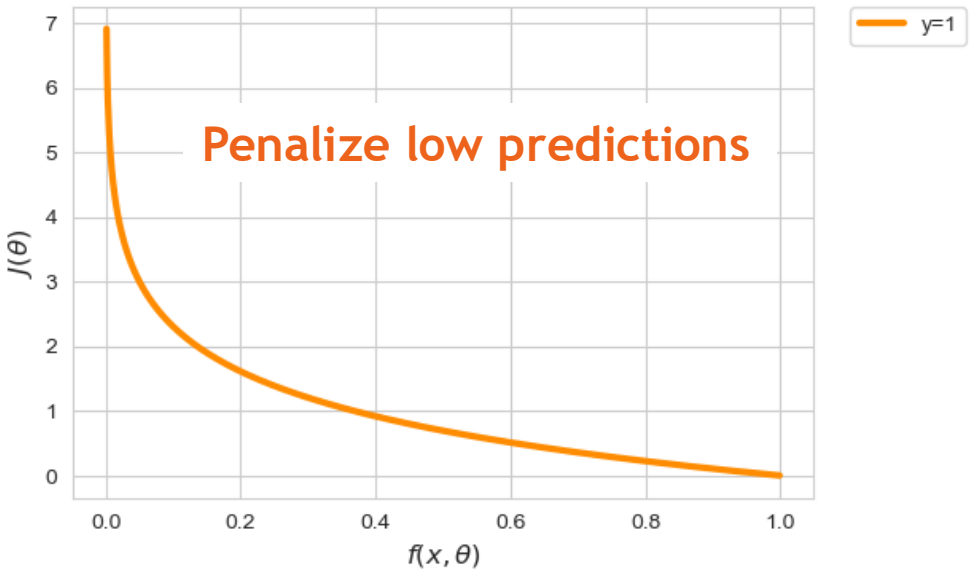
Output layer

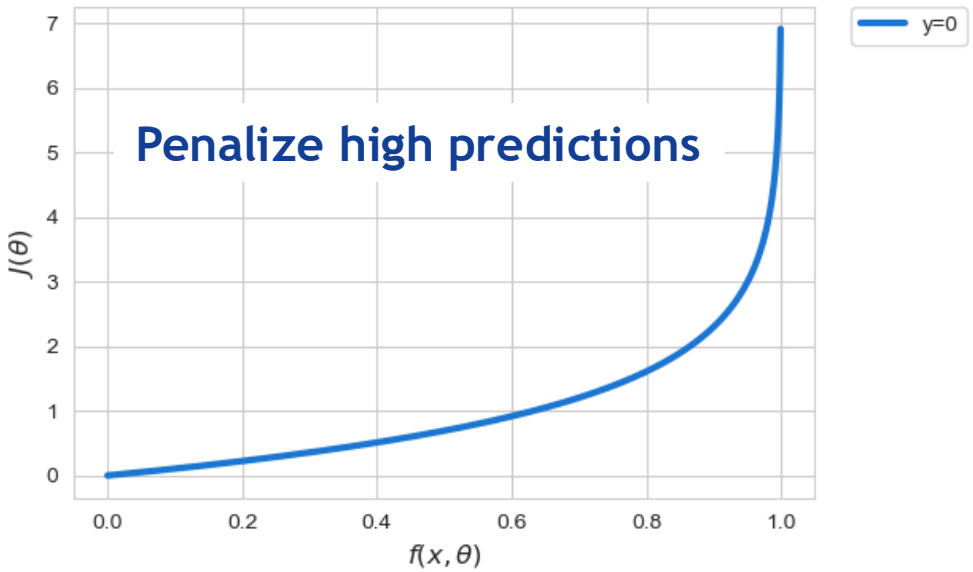
Logistic regression - fitting

$$J(\theta) = -[\frac{1}{n}\sum_{i=1}^{n} y^{(i)} \log\left(f\left(x^{(i)},\theta\right)\right) + (1-y^{(i)})\log(1-f\left(x^{(i)},\theta\right)$$

$y^{(i)} = 1$

$y^{(i)} = 0$

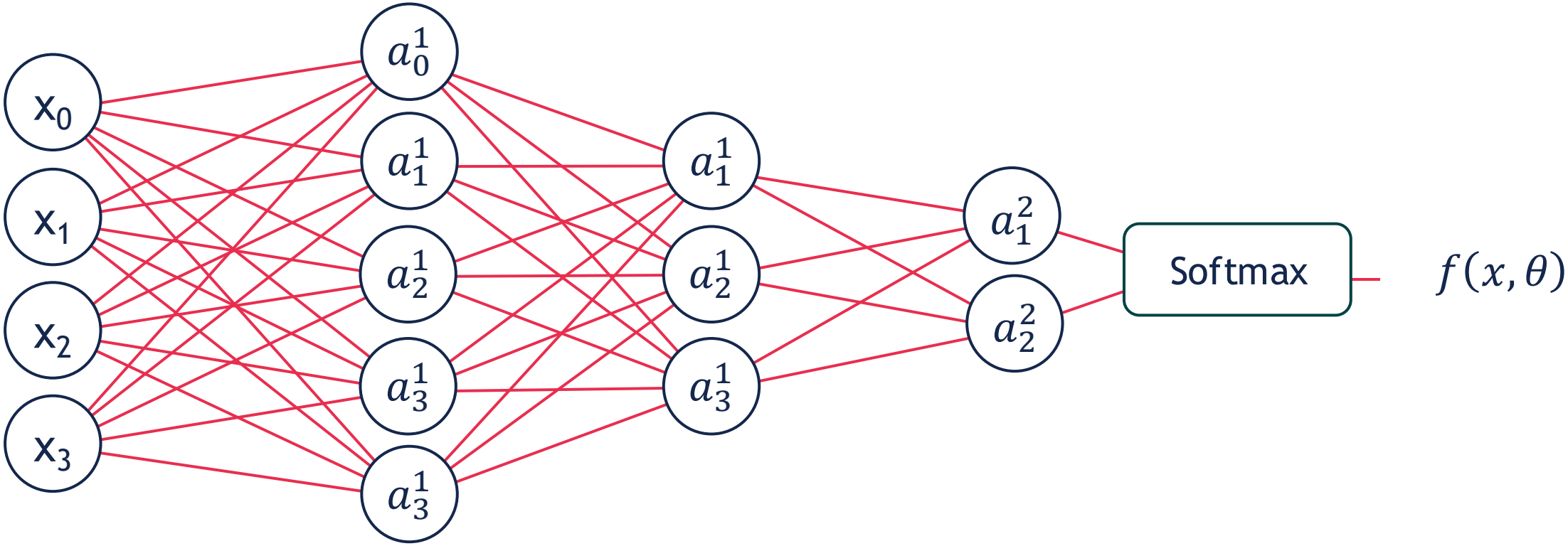




Backpropagation

Adjust the weights based on the prediction errors (i.e. gradient descent)

Chain rule → gradients can be computed per layer



Input
(4 features)

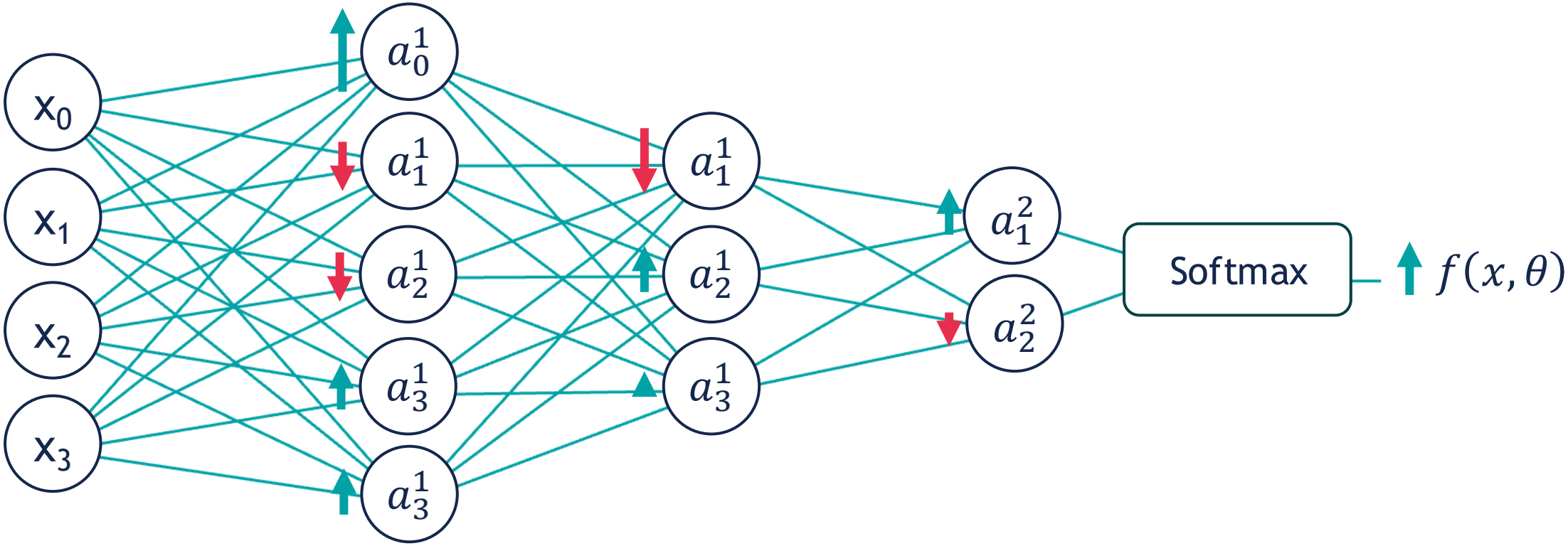
Hidden layers
(5 and 3 nodes)

Output layer

Backpropagation

Adjust the weights based on the prediction errors (i.e. gradient descent)

Chain rule → gradients can be computed per layer

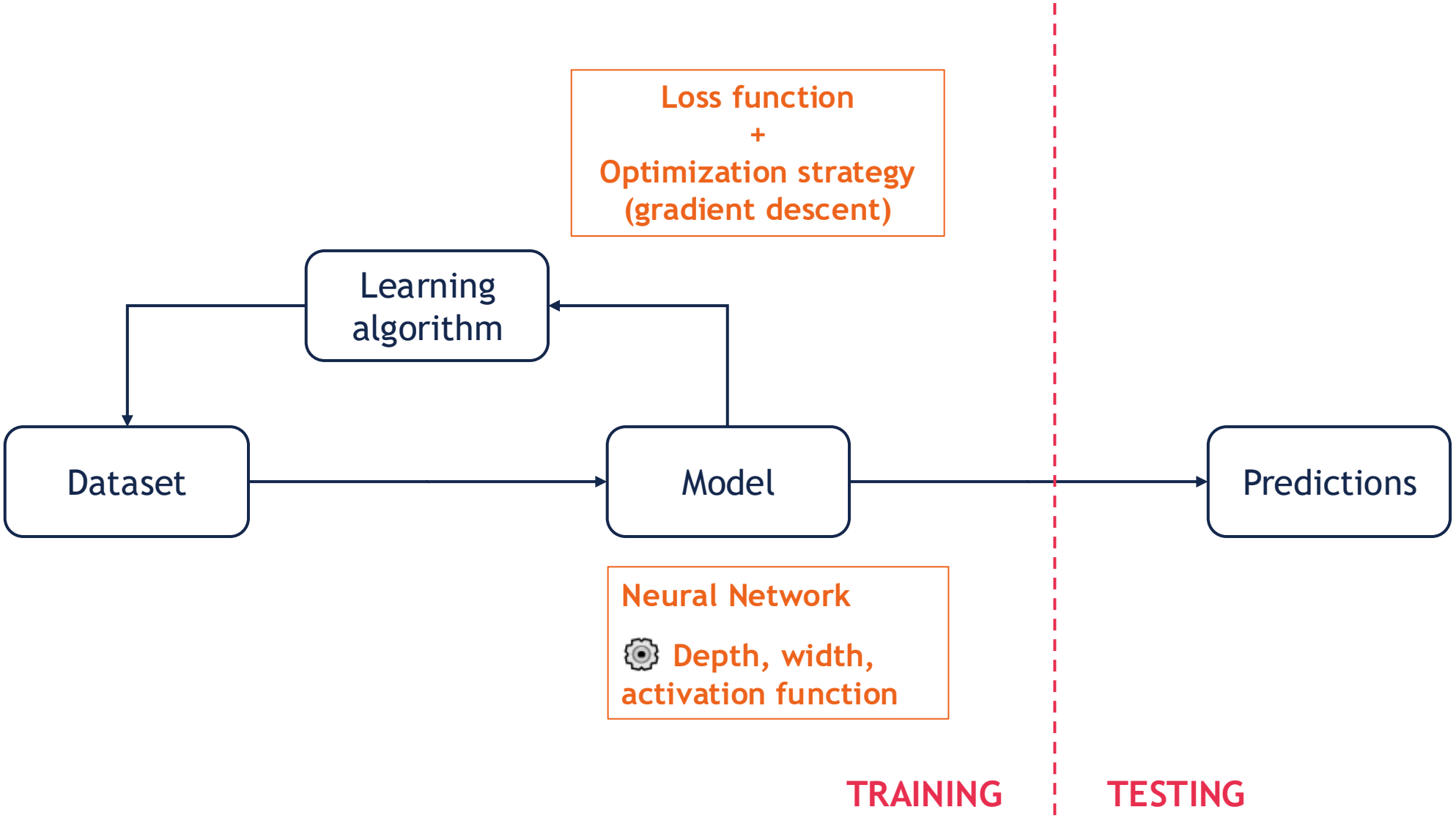


Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

Training a neural network



VIB